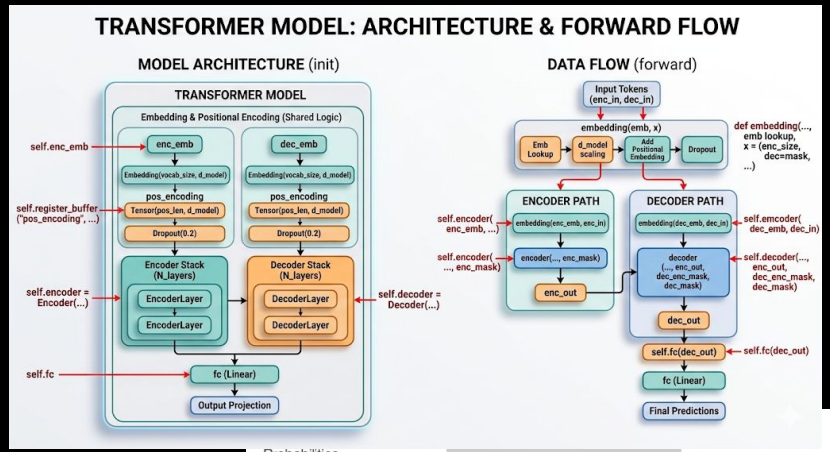
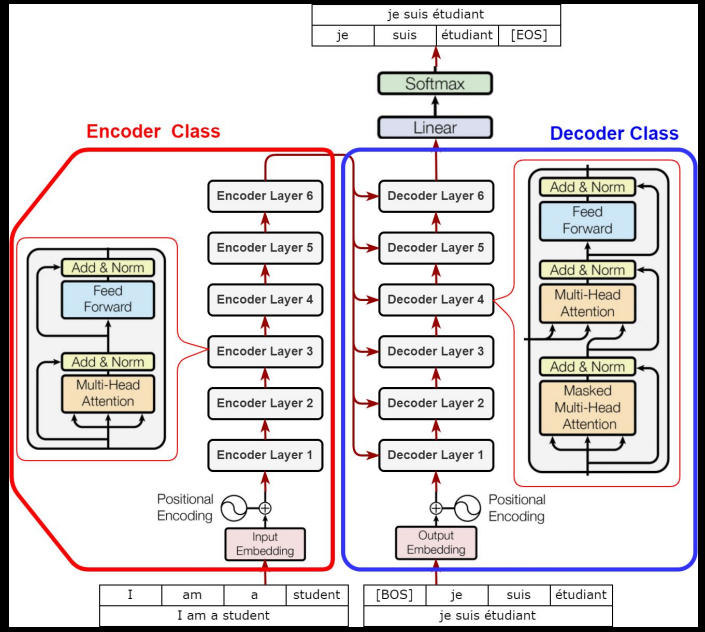
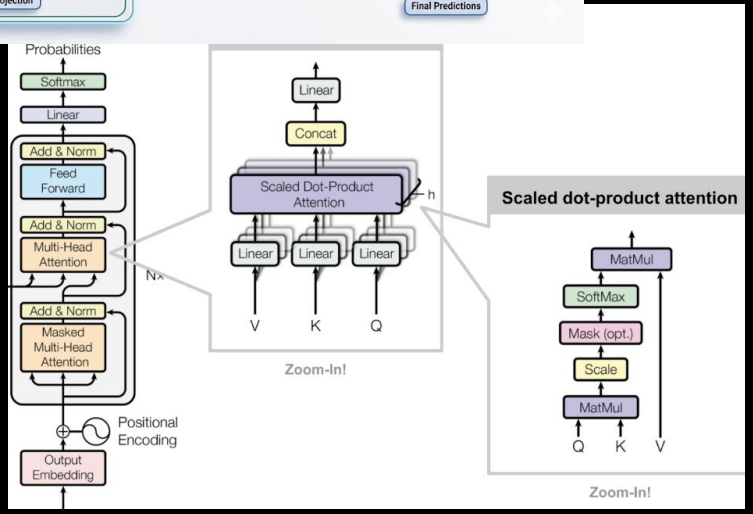
Transformer 코드와 그림들을 연결해서 이해하고 싶다.





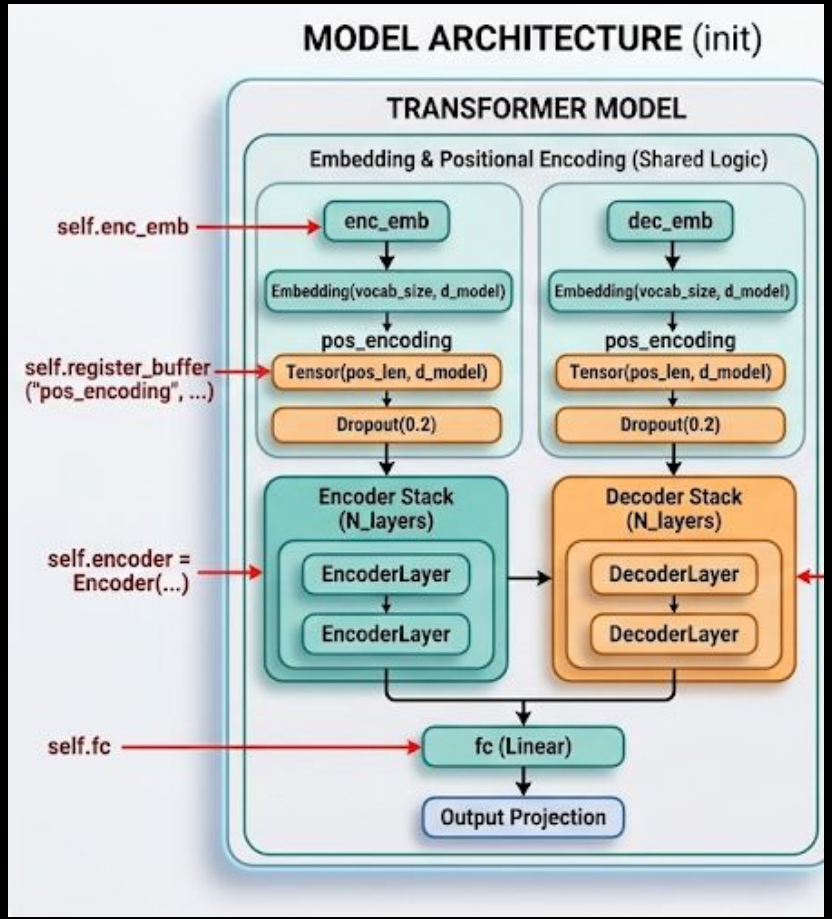


목차

Transformer - 구조
Transformer - 생성자
Transformer - Encoder Stack
Transformer - Encoder Stack - Encoder Layer
Transformer - Encoder Stack - Encoder Layer - Multi Head Attention - split_header
Transformer - Encoder Stack - Encoder Layer - Multi Head Attention - Scaled dot-product
Transformer - Encoder Stack - Encoder Layer - Multi Head Attention - Concat, Linear
Transformer - Encoder Stack - Encoder Layer - FFN

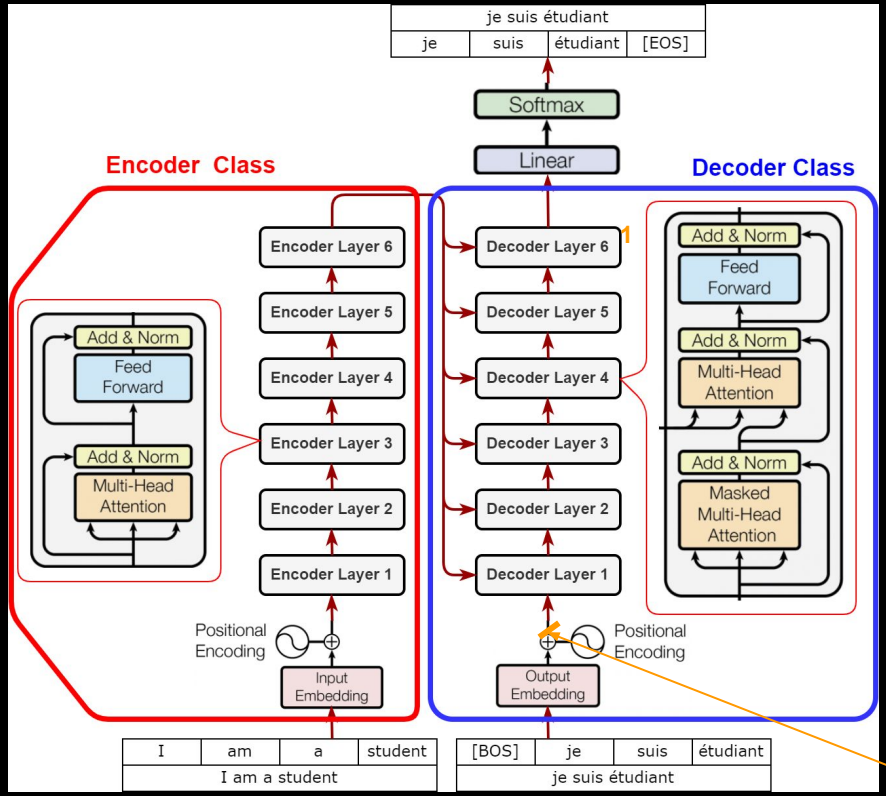
Transformer - Decoder Stack
Transformer - Decoder Stack - Decoder Layer

Transformer - 구조




```python
class Transformer(nn.Module):

    def __init__(self, n_layers, d_model, n_heads,
                 d_ff, src_vocab_size, tgt_vocab_size, pos_len, dropout=0.2):

        self.enc_emb = nn.Embedding(src_vocab_size, d_model)
        self.dec_emb = nn.Embedding(tgt_vocab_size, d_model)

        self.register_buffer("pos_encoding", torch.tensor(
                                                 positional_encoding(pos_len, d_model),
                                                 dtype=torch.float32))
        self.do = nn.Dropout(dropout)

        self.encoder = Encoder(n_layers, d_model, n_heads, d_ff, dropout)
        self.decoder = Decoder(n_layers, d_model, n_heads, d_ff, dropout)

        self.fc = nn.Linear(d_model, tgt_vocab_size)
```

Transformer - 생성자



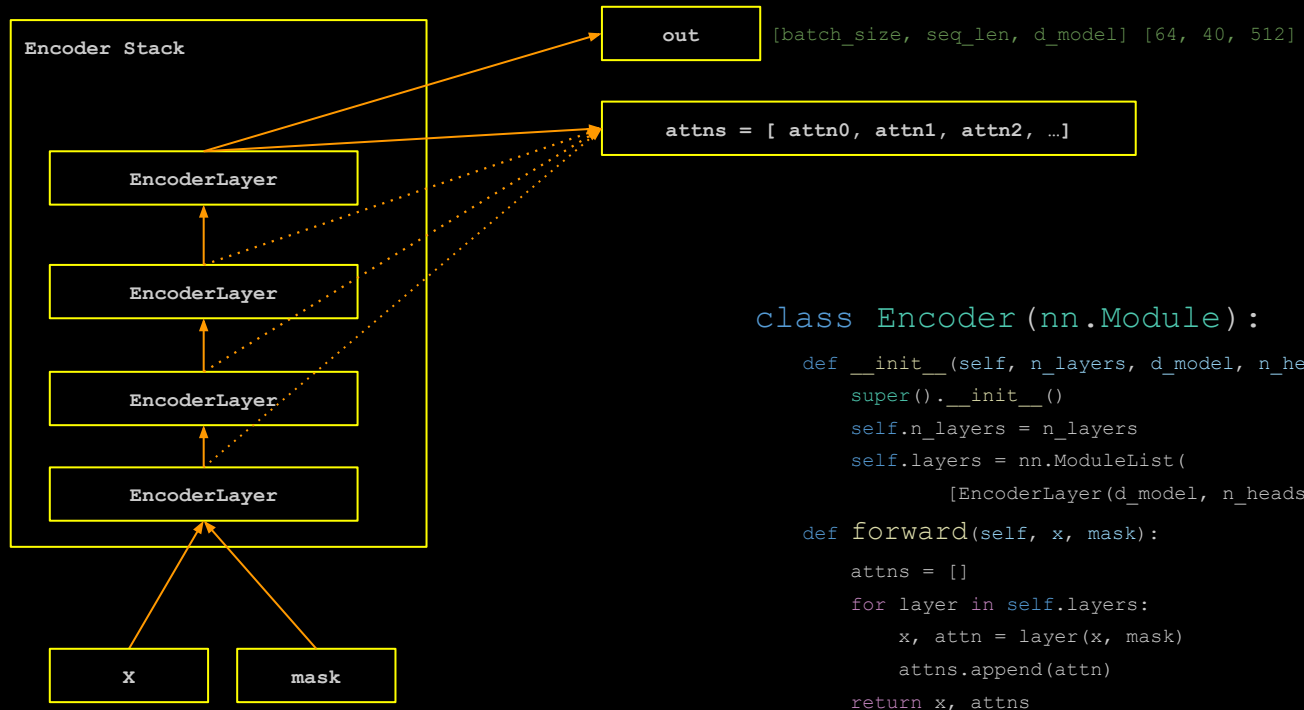

```
transformer = Transformer(
    n_layers=6, d_model=512,
    n_heads=8, d_ff=1024,
    src_vocab_size = len(vocab),
    tgt_vocab_size = len(vocab),
    pos_len=40)
```

번호 (No.)	인자 (Argument)	설명 (Description)
1	`n_layers`	Encoder와 Decoder에서 사용할 레이어의 수입니다.
2	`d_model`	모델 내부 레이어의 차원 수(임베딩 벡터의 크기)입니다.
3	`n_heads`	멀티 헤드 어텐션에서 사용할 헤드의 수입니다.
4	`d_ff`	피드 포워드(Feed-forward) 네트워크 은닉층의 차원 수입니다.
5	`src_vocab_size`	소스 문장(Encoder 입력)의 어휘 사전 크기입니다.
6	`tgt_vocab_size`	타겟 문장(Decoder 입력/출력)의 어휘 사전 크기입니다.
7	`pos_len`	포지셔널 인코딩(Positional Encoding)을 위한 최대 문장 길이입니다.
8	`dropout`	드롭아웃(Dropout) 비율입니다 (기본값: 0.2).

* vocab, 어휘 사전, { 토큰: id } 목록

[batch_size, seq_len, d_model] [64, 40, 512]

Transformer - Encoder Stack




```python
class Encoder(nn.Module):
    def __init__(self, n_layers, d_model, n_heads, d_ff, dropout):
        super().__init__()
        self.n_layers = n_layers
        self.layers = nn.ModuleList(
                [EncoderLayer(d_model, n_heads, d_ff, dropout) for _ in range(n_layers)])
    def forward(self, x, mask):
        attns = []
        for layer in self.layers:
            x, attn = layer(x, mask)
            attns.append(attn)
        return x, attns
```

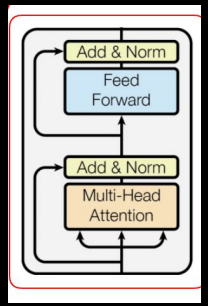
x = batch_size, seq_len, d_model] [64, 40, 512]

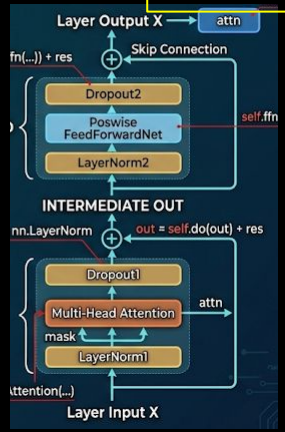
Transformer - Encoder Stack - Encoder Layer

[batch_size, seq_len, d_model] [64, 40, 512]

out, attn

out, attn





x

x

mask

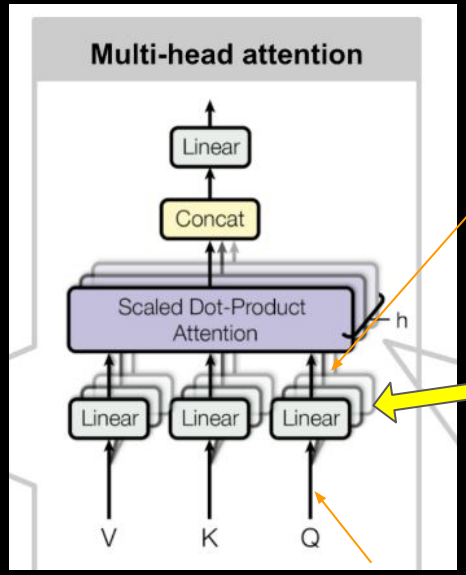
[64, 1, 1, 40]

[batch_size, seq_len, d_model] [64, 40, 512]

```
class EncoderLayer(nn.Module):
    def __init__(self, d_model, n_heads, d_ff, dropout):
        super().__init__()
        self.mha = MultiHeadAttention(d_model, n_heads)
        self.ffn = PoswiseFeedForwardNet(d_model, d_ff)
        self.norm1 = nn.LayerNorm(d_model, eps=1e-6)
        self.norm2 = nn.LayerNorm(d_model, eps=1e-6)
        self.do = nn.Dropout(dropout)

    def forward(self, x, mask):
        res = x
        out, attn = self.mha(self.norm1(x), self.norm1(x), self.norm1(x), mask)
        out = self.do(out) + res

        res = out
        out = self.do(self.ffn(self.norm2(out))) + res
        return out, attn
```

batch_size=64

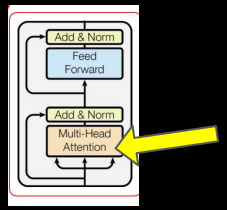
Transformer - Encoder Stack - Encoder Layer - Multi Head Attention - split_header



multi head를 만든다. K, Q, V 차원을 늘린다.

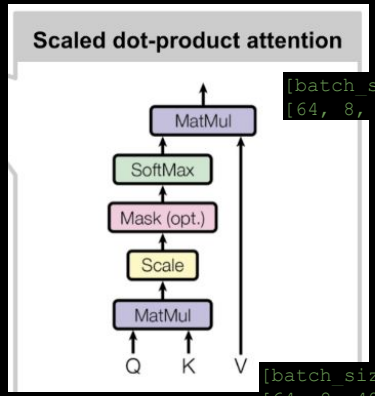
[batch_size, heads, seq_len, d_model]
(64, 8, 40, 64)

[batch_size, seq_len, d_model]
(64, 40, 512)




```python
class MultiHeadAttention(nn.Module):
    def forward(self, Q, K, V, mask=None):
        WQ, WK, WV = self.W_q(Q), self.W_k(K), self.W_v(V)
        Q_split = self.split_heads(WQ)
        K_split = self.split_heads(WK)
        V_split = self.split_heads(WV)
        out, attn = self.scaled_dot_product_attention(
                                        Q_split, K_split, V_split, mask)
        combined_out = self.combine_heads(out)
        return self.linear(combined_out), attn

    def split_heads(self, x):
        bsz, seq_len, _ = x.size() # [batch_size, seq_len, d_model] -> (64, 40, 512)
        x_view = x.view(bsz, seq_len, self.num_heads, self.depth) # [batch_size, seq_len, heads, depth]
        out = x_view.permute(0, 2, 1, 3) # [batch_size, heads, seq_len, d_model] -> (64, 8, 40, 64)
        return out
```

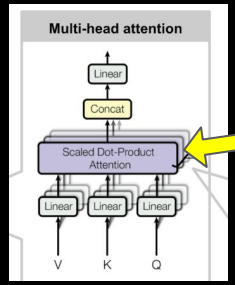
Transformer - Encoder Stack - Encoder Layer - Multi Head Attention - Scaled dot-product attention



[batch_size, num head, seq_len, depth]
[64, 8, 40, 64]

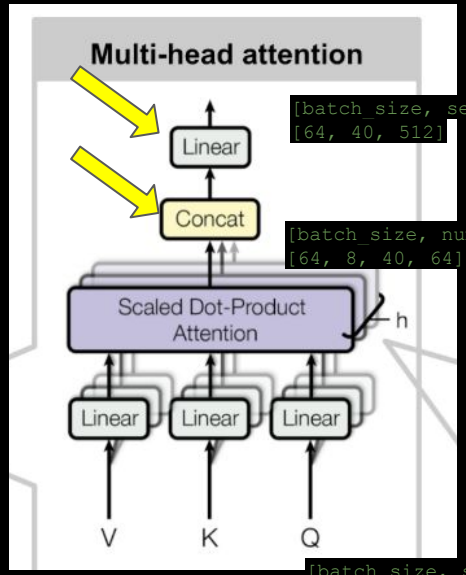
[batch_size, num head, seq_len, depth]
[64, 8, 40, 64]

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$




```python
class MultiHeadAttention(nn.Module):
    def forward(self, Q, K, V, mask=None):
        WQ, WK, WV = self.W_q(Q), self.W_k(K), self.W_v(V)
        Q_split = self.split_heads(WQ)
        K_split = self.split_heads(WK)
        V_split = self.split_heads(WV)
        out, attn = self.scaled_dot_product_attention(
                                        Q_split, K_split, V_split, mask)
        combined_out = self.combine_heads(out)
        return self.linear(combined_out), attn

    def scaled_dot_product_attention(self, Q, K, V, mask=None):
        K_t = K.transpose(-1, -2) # K 마지막 두 차원을 맞바꿈. [64,8,64,40]
        QK = torch.matmul(Q, K_t) # [64, 8, 40, 40] = [64, 8, 40, 64] * [64, 8, 64, 40]

        d_k = Q.size(-1)          # Q의 마지막 차원 값 : 64
        scaled_qk = QK / math.sqrt(d_k) # [64, 8, 40, 40]

        if mask is not None:
            mask_value = mask * -1e9
            scaled_qk += mask_value # 매우 작은 음수 더하기 -> 이후 softmax 결과값이 0으로
수렴

        attentions = F.softmax(scaled_qk, dim=-1) # 0~1 사이의 확률값, 집중도 # [64, 8, 40, 40]

        out = torch.matmul(attentions, V) # [64, 8, 40, 64] = [64, 8, 40, 40] * [64,8,40,64]
        return out, attentions
```

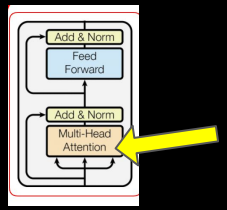
Transformer - Encoder Stack - Encoder Layer - Multi Head Attention - Concat, Linear



[batch_size, seq_len, d_model]
[64, 40, 512]

[batch_size, num_head, seq_len, depth]
[64, 8, 40, 64]

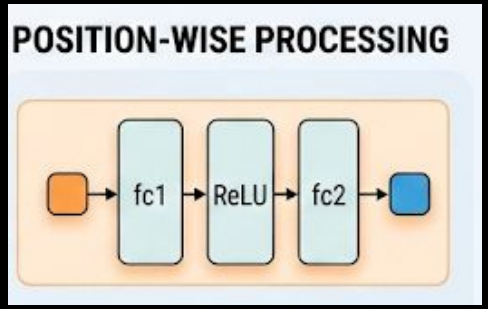
[batch_size, seq_len, d_model]
[64, 40, 512]



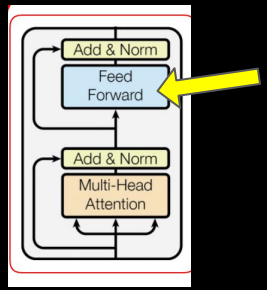

```python
class MultiHeadAttention (nn.Module):
    def forward(self, Q, K, V, mask=None):
        WQ, WK, WV = self.W_q(Q), self.W_k(K), self.W_v(V)
        Q_split = self.split_heads(WQ)
        K_split = self.split_heads(WK)
        V_split = self.split_heads(WV)
        out, attn = self.scaled_dot_product_attention(
                                        Q_split, K_split, V_split, mask)
        combined_out = self.combine_heads(out)
        return self.linear(combined_out), attn


    def combine_heads(self, x):
        x_permute = x.permute(0, 2, 1, 3) # [64,8,40,64] -> [64,40,8,64]

        x_contig = x_permute.contiguous()   # 메모리 주소를 연속으로 재배치해서
        bsz, num_heads, seq_len, depth = x.size()
        out = x_contig.view(bsz, seq_len, self.d_model) # 차원 축소 [64, 40,512]]
        return out
```

Transformer - Encoder Stack - Encoder Layer - FFN

[batch_size, seq_len, d_model]
[64, 40, 512]

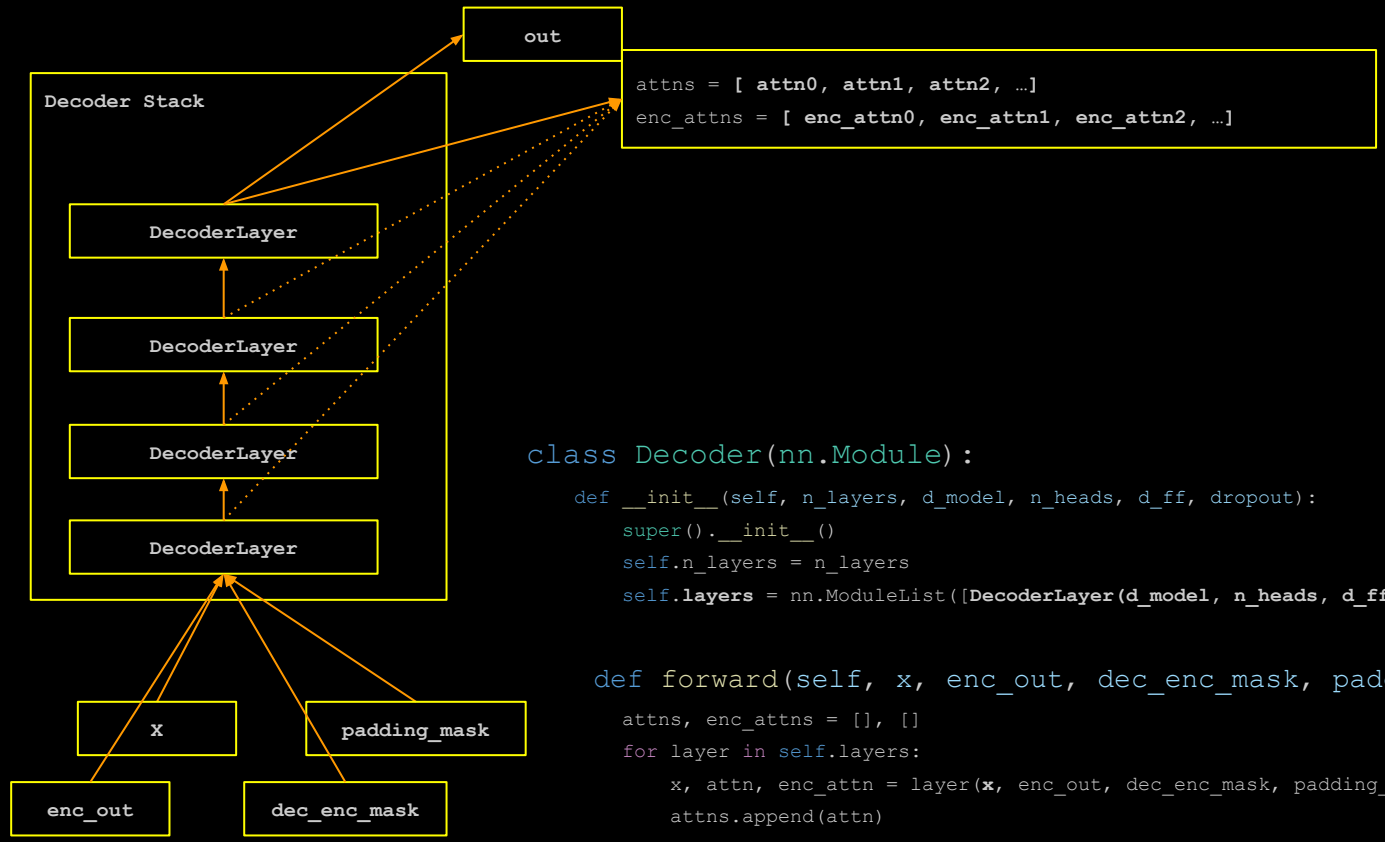


[batch_size, seq_len, d_model]
[64, 40, 512]




```python
class EncoderLayer(nn.Module):
    def __init__(self, d_model, n_heads, d_ff, dropout):
        super().__init__()
        self.mha = MultiHeadAttention(d_model, n_heads)
        self.ffn = PoswiseFeedForwardNet(d_model, d_ff)
        self.norm1 = nn.LayerNorm(d_model, eps=1e-6),
        self.norm2 = nn.LayerNorm(d_model, eps=1e-6)
        self.do = nn.Dropout(dropout)

class PoswiseFeedForwardNet (nn.Module):
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.fc1 = nn.Linear(d_model, d_ff)
        self.fc2 = nn.Linear(d_ff, d_model)

    def forward(self, x):
        return self.fc2(F.relu(self.fc1(x)))
```

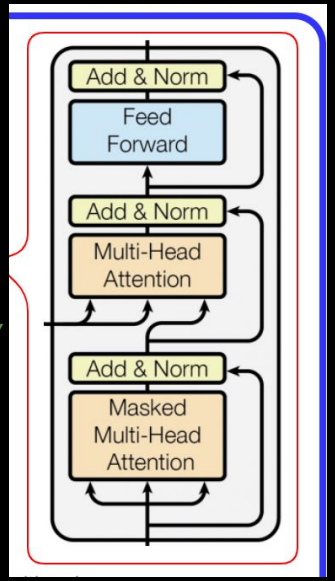
Transformer - Decoder Stack




```python
class Decoder(nn.Module):
    def __init__(self, n_layers, d_model, n_heads, d_ff, dropout):
        super().__init__()
        self.n_layers = n_layers
        self.layers = nn.ModuleList([DecoderLayer(d_model, n_heads, d_ff, dropout) for _ in range(n_layers)])

    def forward(self, x, enc_out, dec_enc_mask, padding_mask):
        attns, enc_attns = [], []
        for layer in self.layers:
            x, attn, enc_attn = layer(x, enc_out, dec_enc_mask, padding_mask)
            attns.append(attn)
            enc_attns.append(enc_attn)
        return x, attns, enc_attns
```

Transformer - Decoder Stack - Decoder Layer

out = [batch_size, seq_len, d_model],
[64, 40, 512]



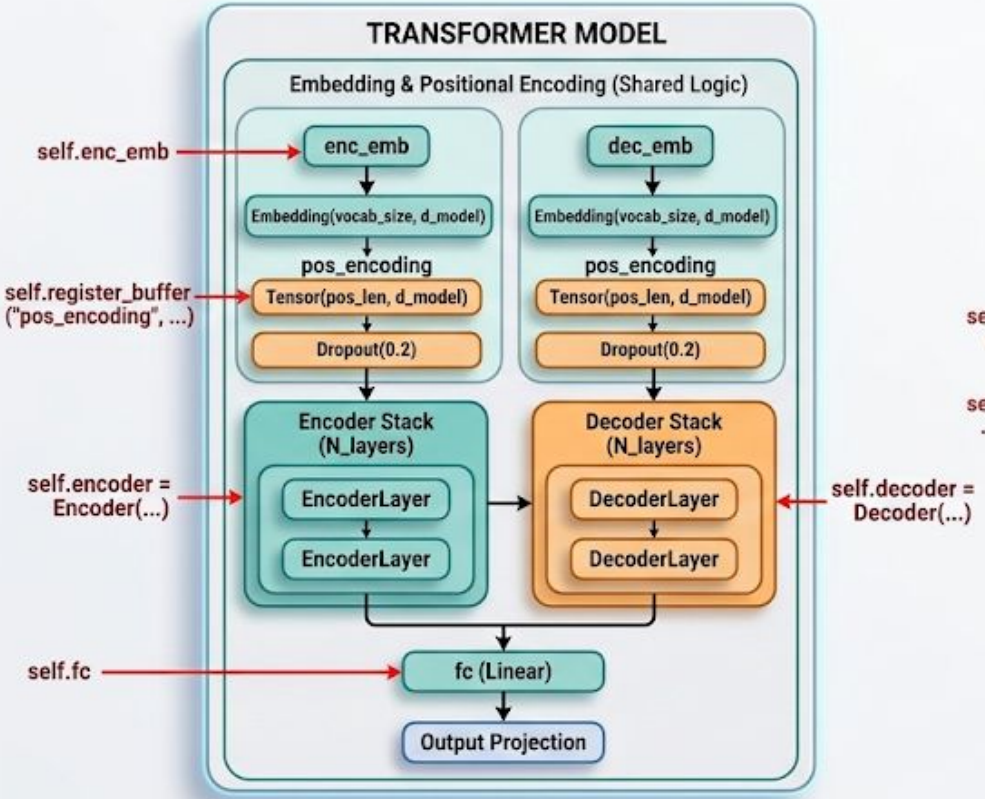
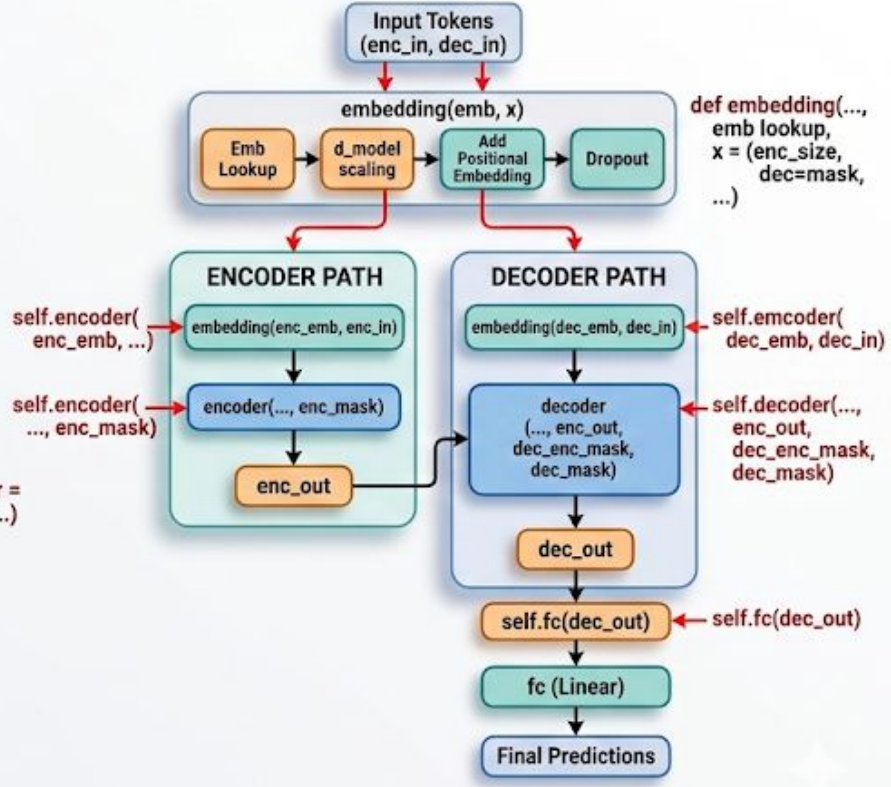
enc_out =
[batch_size, seq_len, d_model],
[64, 40, 512]

x = [batch_size, seq_len, d_model],
[64, 40, 512]

```
class DecoderLayer(nn.Module):
    def __init__(self, d_model, num_heads, d_ff, dropout):
        super().__init__()
        self.mha1 = MultiHeadAttention(d_model, num_heads)
        self.mha2 = MultiHeadAttention(d_model, num_heads)
        self.ffn = PoswiseFeedForwardNet(d_model, d_ff)
        self.norm1 = nn.LayerNorm(d_model, eps=1e-6)
        self.norm2 = nn.LayerNorm(d_model, eps=1e-6),
        self.norm3 = nn.LayerNorm(d_model, eps=1e-6)
        self.do = nn.Dropout(dropout)

    def forward(self, x, enc_out, dec_enc_mask, padding_mask):
        res = x
        out, attn1 = self.mha1(self.norm1(x), self.norm1(x), self.norm1(x), padding_mask)
        out = self.do(out) + res

        res = out
        out, attn2 = self.mha2(self.norm2(out), enc_out, enc_out, dec_enc_mask)
        out = self.do(out) + res

        res = out
        out = self.do(self.ffn(self.norm3(out))) + res
        return out, attn1, attn2
```

TRANSFORMER MODEL: ARCHITECTURE & FORWARD FLOW

MODEL ARCHITECTURE (init)



DATA FLOW (forward)